

Sujet Grand Oral NSI : Pourquoi AlphaZero représente-t-il une rupture dans l

► SommaireSauter à une section ▼

Bonjour. Je vais vous parler d'un événement qui a marqué l'histoire de l'**informatique** : la victoire d'AlphaZero contre Stockfish en deux mille dix-sept.

Le **sujet** est précis : pourquoi AlphaZero représente-t-il une rupture dans l'intelligence artificielle appliquée aux jeux de stratégie ? Cette **question** m'intéresse parce qu'elle se trouve à la frontière du **programme** de spécialité **NSI en terminale** : elle mobilise les notions d'apprentissage automatique, de réseaux de neurones, et plus largement la distinction entre **programmation impérative** et **apprentissage statistique**.

Avant deux mille dix-sept, tous les **programmes** d'échecs reposaient sur le même paradigme : un algorithme de recherche arborescente comme alpha-bêta, guidé par une fonction d'évaluation écrite par des experts humains. Cette approche, héritée de Claude Shannon en 1950, avait connu cinquante ans d'améliorations progressives. AlphaZero, développé par DeepMind, a battu Stockfish, alors le meilleur **programme** au monde, en quatre **heures** d'entraînement, sans connaissance humaine initiale. C'est ce que je vais analyser ici.

Je procéderai en trois étapes. D'abord, je rappellerai brièvement l'approche classique pour mesurer ce qui change. Ensuite, je présenterai l'architecture d'AlphaZero et son fonctionnement. Enfin, je discuterai la nature de cette rupture et ses **limites**.

L'approche classique : minimax et fonctions d'évaluation

Pour comprendre ce qu'AlphaZero a changé, il faut d'abord comprendre ce qu'il a remplacé. Pendant cinquante ans, les **programmes** d'échecs ont fonctionné sur deux composants. D'un côté, un algorithme de recherche : minimax avec élagage alpha-bêta, qui explore un **arbre** de coups jusqu'à une profondeur donnée. De l'autre, une fonction d'évaluation : une formule qui donne un score à une position non terminale, fondée sur la valeur des pièces, le contrôle du centre, la sécurité du roi, et d'autres critères codés à la main.

Le meilleur **exemple** de cette approche est Stockfish, **programme** open source développé depuis deux mille huit. Il évalue environ deux cents millions de positions par seconde. Sa fonction d'évaluation est le fruit de quinze années de réglages fins par une communauté de centaines de développeurs. C'est un sommet de **programmation impérative** : chaque ligne de code dit explicitement à la machine ce qu'elle doit faire et pourquoi.

L'avantage de cette approche est l'**explicabilité** : on peut comprendre pourquoi le **programme** joue tel coup. La fonction d'évaluation est lisible, l'**arbre** exploré est traçable. C'est aussi sa **limite** principale : la fonction d'évaluation reflète les intuitions humaines, donc elle hérite des biais et des angles morts des experts qui l'ont écrite.

En deux mille dix-sept, le moteur Stockfish dépassait le niveau de tous les joueurs humains, y compris le champion du monde. On pouvait imaginer que le plafond avait été atteint, et que les progrès futurs seraient incrémentaux. C'est cette croyance qu'AlphaZero a brisée.

AlphaZero : architecture et fonctionnement

AlphaZero a été présenté par DeepMind, filiale de Google, dans un article publié dans Science en deux mille dix-huit. Le **programme** repose sur trois composants nouveaux par rapport à Stockfish.

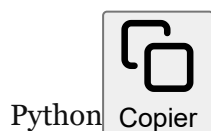
Premièrement, un **réseau de neurones** profond. Ce réseau prend en entrée une position d'échecs (encodée sur un tenseur de dimensions $8 \times 8 \times \text{planes}$), et produit deux sorties. La première est un score de valeur, entre moins un et plus un, qui estime qui va gagner depuis cette position. La seconde est une distribution de probabilité sur les coups possibles, qui indique lesquels méritent d'être explorés en priorité.

Deuxièmement, un algorithme de recherche appelé **Monte Carlo Tree Search**, ou MCTS. Contrairement au minimax, qui explore l'**arbre** de manière déterministe et exhaustive, MCTS explore de manière sélective et stochastique. Il visite préférentiellement les nœuds que le réseau de neurones juge prometteurs, accumule des statistiques de victoire et de défaite par simulation, et retourne le coup le plus visité.

Troisièmement, un mécanisme d'**apprentissage par renforcement**. AlphaZero joue contre lui-même, des millions de parties. À chaque partie, il observe le résultat (victoire, défaite, nulle) et ajuste les poids de son réseau de neurones pour mieux prédire les positions gagnantes la prochaine fois. C'est une boucle d'auto-amélioration : le réseau guide MCTS, MCTS produit des **données**, ces **données** entraînent le réseau, et le cycle recommence.

Le résultat est spectaculaire. En quatre **heures** d'entraînement sur du matériel spécialisé (5 000 TPUs), AlphaZero atteint un niveau supérieur à Stockfish. Lors du match officiel, sur cent parties, AlphaZero a gagné vingt-huit fois, n'a jamais perdu, et a fait soixante-douze nulles. Aucune connaissance humaine n'a été injectée : AlphaZero a redécouvert seul les principes des ouvertures, des structures de pions, des sacrifices positionnels. Mieux : il a inventé des stratégies que les commentateurs humains ont qualifiées de profondément originales.

Voici une représentation simplifiée du cœur de l'algorithme en pseudo-code Python.



```
def alphazero_self_play():
    reseau = ReseauNeurones()
    for partie in range(millions):
        position = position_initiale()
        historique = []
        while not position.est_terminee():
            distribution = mcts(position, reseau, simulations=800)
            coup = echantillonner(distribution)
            historique.append((position, distribution))
            position.joue(coup)
        resultat = position.resultat()
        for pos, dist in historique:
            reseau.entrainer(pos, dist, resultat)
```

Le cœur de ce code est la fonction MCTS, qui utilise le réseau pour orienter sa recherche, et la fonction d'entraînement, qui ajuste les poids du réseau à partir des résultats des parties. Aucune connaissance des échecs n'est codée explicitement : on fournit les règles du jeu et un signal de récompense, et le réseau découvre seul le reste.

La nature de la rupture et ses limites

En quoi AlphaZero représente-t-il une rupture épistémologique, et pas seulement une amélioration ? La réponse tient en trois points.

Premièrement, c'est un changement de **paradigme**. L'approche classique relève de la **programmation impérative** : le développeur dit à la machine comment résoudre le problème. AlphaZero relève de l'**apprentissage automatique** par renforcement : le développeur dit à la machine quoi atteindre, et la machine découvre seule comment. Ces deux paradigmes coexistent dans le **programme** de spécialité **NSI**, mais AlphaZero est l'un des **exemples** les plus spectaculaires du second.

Deuxièmement, c'est une rupture sur la **généralité**. Stockfish ne sait jouer qu'aux échecs : sa fonction d'évaluation est spécifique à ce jeu. AlphaZero, avec exactement le même code, peut apprendre le Go, le Shogi, ou n'importe quel jeu à information complète. Il suffit de lui fournir les règles. C'est une généralisation impressionnante. Quelques mois plus tard, le **système** dérivé MuZero apprenait des jeux dont on ne lui donnait même pas les règles. La portée de cette **méthode** dépasse largement les échecs.

Troisièmement, c'est une rupture sur l'**explicabilité**. Stockfish est explicable : pour chaque coup, on peut retracer l'**arbre** et identifier les variantes qui ont conduit au choix. AlphaZero est une boîte noire : son réseau de neurones a des dizaines de millions de paramètres, et personne ne peut dire en français pourquoi il joue tel coup. Cette opacité est l'un des grands **enjeux** philosophiques de l'**informatique** moderne. Elle pose la **question** : peut-on accepter des décisions très performantes mais non explicables ? Dans le contexte des échecs, c'est anecdotique. Dans le contexte médical ou judiciaire, c'est un problème majeur.

Les **limites** d'AlphaZero sont également significatives. Premièrement, l'entraînement coûte cher : 5 000 TPUs pendant plusieurs **heures**, soit plusieurs millions de dollars d'infrastructure. Cette barrière à l'entrée concentre la recherche dans quelques grandes entreprises. Deuxièmement, AlphaZero reste lent

en termes de positions évaluées : environ soixante mille par seconde, contre deux cents millions pour Stockfish. Sa supériorité vient de la qualité de l'évaluation, pas du volume de calcul. Troisièmement, le **système** ne sait pas généraliser hors de son domaine d'entraînement : un AlphaZero entraîné aux échecs ne saura pas spontanément jouer aux dames.

Conclusion

Pour répondre à ma **question** initiale, AlphaZero représente une rupture dans l'intelligence artificielle parce qu'il abandonne la **programmation explicite** au profit de l'**apprentissage par renforcement**, et qu'il démontre que cette approche peut surpasser cinquante ans d'expertise humaine en quatre **heures**. Cette rupture n'est pas seulement quantitative : elle change la nature de ce que signifie « programmer » une intelligence artificielle.

L'ouverture la plus profonde, c'est de constater que les principes d'AlphaZero ont été transposés ensuite à des problèmes scientifiques majeurs. AlphaFold, dérivé direct d'AlphaZero, a résolu en deux mille vingt le problème du repliement des protéines, qui était ouvert depuis cinquante ans. Cette percée a valu le prix Nobel de chimie deux mille vingt-quatre à ses auteurs. Les échecs ont donc été le laboratoire d'une révolution **informatique** dont l'impact dépasse de loin le cadre des jeux.

Je vous remercie pour votre attention et je suis prêt à répondre à vos **questions**.